# de1
## Joining Tables
**completed on 10/15/2026, 3:18:29 PM**

### 1 Countries + Geoloc

Let's start by **joining** the **countries** and **geoloc** tables. The geoloc table has **latitude** and **longitude** values for the center of each country.

Enter the following **NATURAL JOIN** query to see how it combines the data into a single table:

```
SELECT * FROM countries NATURAL JOIN geoloc
```

| ccd | country | continent | population | area | latitude | longitude |
|---|---|---|---|---|---|---|
| AD | Andorra | Europe | 77000 | 468 | 42.5462 | 1.6015 |
| AE | United Arab Emira | Asia | 9631000 | 83600 | 23.424 | 53.8478 |
| AF | Afghanistan | Asia | 37172000 | 652090 | 33.9391 | 67.7099 |
| AG | Antigua and Barbu | North America | 96000 | 442 | 17.0608 | -61.7964 |
| AI | Anguilla | North America | 15000 | 96 | 18.2205 | -63.0686 |

Correct!

### 2 Southern Hemisphere

For each country, we now have all the columns from both tables. Awesome! This works because **NATURAL JOIN** is able to use the **ccd** column to match up the two tables.

In the results tab, switch over to **picture mode** and notice that all countries are selected. Let's add a `WHERE` clause to select only the countries that are **south** of the **equator**. In other words, countries where **latitude < 0**.



Interesting! The equator looks off balance without showing Antarctica.

### 3 Eastern Countries

How would you select all the countries **east** of the **prime meridian**?

```
SELECT * FROM countries
NATURAL JOIN geoloc
WHERE longitude > 0
```

Correct!

### 4 Coastal Countries

Now let's join **countries** with the **coastlines** tables.

Write a query to select only the countries that have a coastline greater than 0.

```
SELECT * FROM countries
NATURAL JOIN coastlines
WHERE coast_km > 0
```

Correct!

### 5 Different Column Names

The **NATURAL JOIN** clause is easy to use, but it only works if the corresponding columns are **named the same** in both tables.

Take a look at the **shapes** and the **perimeters** tables. Notice that the IDs are called **id** in the shapes table, but are called **shape_id** in the perimeters table.

Since the columns have different names, the natural join will not work the way we want. Try it out and **see what happens**:

| id | shape | color | dots | xloc | yloc | scale | shape_id | perimeter |
|---|---|---|---|---|---|---|---|---|
| 0 | circle | orange | 5 | 420 | 100 | 0.6 | 7 | 636 |
| 0 | circle | orange | 5 | 420 | 100 | 0.6 | 8 | 312 |
| 0 | circle | orange | 5 | 420 | 100 | 0.6 | 9 | 318 |
| 0 | circle | orange | 5 | 420 | 100 | 0.6 | 10 | 301 |
| 0 | circle | orange | 5 | 420 | 100 | 0.6 | 11 | 312 |

Boom! Notice the explosion of shape rows.

## 6 Basic JOIN

Since the key column names are different, the **NATURAL JOIN** didn't know how to match rows. As a result, it matched **everything to everything**, meaning it matched every column in the first table with every column in the second table creating a **very large table.** This result is called a **CROSS JOIN** and it's not what we want here.

We want each shape to line up with its corresponding perimeter. To do this, we need to use a basic `JOIN` clause and explicitly state which columns to equate using the `ON` keyword like this:

```
SELECT * FROM shapes JOIN perimeters ON id=shape_id
```

**Try it** and compare the output to the previous result:

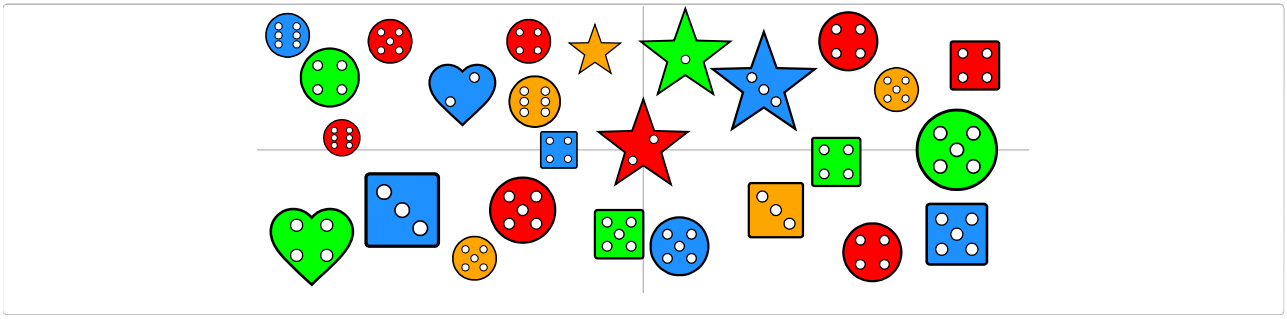
| id | shape | color | dots | xloc | yloc | scale | shape_id | perimeter |
|---|---|---|---|---|---|---|---|---|
| 0 | circle | orange | 5 | 420 | 100 | 0.6 | 0 | 226 |
| 1 | square | blue | 3 | -400 | -100 | 1.2 | 1 | 469 |
| 2 | star | red | 2 | 0 | 0 | 0.7 | 2 | 557 |
| 3 | heart | blue | 2 | -300 | 100 | 0.8 | 3 | 332 |
| 4 | circle | red | 6 | -500 | 20 | 0.5 | 4 | 188 |

Correct!

## 7 Large Perimeters

Let's create a **picture** using this basic **JOIN.**

Write a query to select the shapes that have a **perimeter greater than 400.**



Correct!

8 **Compare and Contrast**

To capture what we learned here, write some notes to describe the **similarities** and **differences** between a **NATURAL JOIN** and a basic **JOIN.**

B I U <> T

Natural join have limits such as if the primary key and foreign key of each table have different name convention, it will not join the table properly. While basic join will join it properly because it automatically inner joins the tables.